

TECHNICAL HANDOVER

Prime Promenade

Website, Admin Dashboard & API — Developer Handover Document

A three-tier lifestyle-destination platform (public marketing & booking site, staff admin dashboard, REST API + database).

Thrissur, Kerala · gym · spa · pool · conference hall · gaming arena · café · pharmacy · study centre · vendor zone.

Prepared for: incoming development team · **Status:** production-ready

Stack: Next.js (React) · Node.js/Express · PostgreSQL · nginx · pm2

Document Contents

1. **Answers to key questions** — SSR/SSG, image optimization, API, forms, security, performance, SEO
2. **Architecture overview**
3. **Folder / project structure**
4. **Deployment process**
5. **API documentation**
6. **Database schema (high level)**
7. **Third-party integrations**
8. **Backup & recovery**
9. **Access, credentials & ownership**
10. **Local development quick-start**

One-line summary. The public website is a **statically generated** Next.js site (served as plain files by nginx); the admin dashboard is a separate Next.js app; both talk to a **Node/Express + PostgreSQL** REST API. Everything runs on a single VPS behind nginx, managed by pm2, with a one-command deploy script.

1 · Answers to Key Questions

Next.js — Front-end framework (React)

Is SSR or SSG being used?

Static Site Generation (SSG) / full static export. The public site uses `output: "export"`, so `next build` emits pre-rendered static HTML/CSS/JS into an `out/` folder that nginx serves directly. There is **no server-side rendering and no Node runtime for the public site** — this makes it fast, cheap, cacheable and highly resilient. Dynamic data (packages, banners, gallery, bookings) is fetched client-side from the API after load. (The admin dashboard is a separate Next.js app run as a Node process; it is an authenticated SPA, not SSR-critical.)

Is image optimization enabled?

Next.js's **built-in** image optimizer is **disabled** (`images: { unoptimized: true }`) — this is **required** for a static export (there is no server to transform images on the fly). Instead, images are **optimized manually at build/authoring time**: source photos are resized to a sensible max width and re-encoded to WebP; the home hero, logos and section images use lightweight variants, with `loading="lazy"` on below-the-fold images and `fetchpriority` hints for the LCP hero. See §7 Performance.

Node.js — Backend / API

How is the API structured?

A **RESTful Express (TypeScript) API**, organised by domain module under `backend/src/modules/*`. Routes are split into two groups:

- **Public routes** — consumed by the website. They return the **raw JSON shape** the site expects (no wrapper), so the front-end contract is stable.
- **Admin routes** — JWT-guarded. They return a consistent envelope `{ success, message, data, meta }`.

Common CRUD is generated from a central **resource registry** (`list/get/create/update/status/delete`), so adding an admin resource is a config entry rather than new boilerplate.

Is there API documentation?

Yes — **interactive Swagger / OpenAPI UI** served by the API at `/api/docs`, with the raw spec at `/api/docs.json`. See §5.

How are forms, bookings and enquiries handled?

Every public form (pool / spa / hall / gym / course / contact / vendor) **POSTs to a public endpoint**. The request is **validated with Zod** (and again by shared front-end validators), then persisted to PostgreSQL via Prisma. Bookings and enquiries are stored as records that staff manage in the admin dashboard; several get a **human-readable reference ID** (e.g. `P00L-00001`, `HALL-00001`, `SPA-00001`, `ENR-00001`). Bookings are **enquiry-style — no online payment is taken**; the admin confirms them. The pool module additionally holds authoritative availability & pricing logic.

PostgreSQL — Database

How are environment variables and API keys stored?

In a `backend/.env` file on the server that is **never committed to git**. A documented `.env.example` lists every variable. Front-end apps only receive **build-time public variables** (prefixed `NEXT_PUBLIC_`) — no secrets ship to the browser. See §9.

What security measures protect the APIs?

Layered defenses (details in §1 table & §9): **JWT** access+refresh auth on all admin routes, **helmet** security headers, a **CORS allow-list**, **HPP** (HTTP-parameter-pollution) protection, **rate limiting**, **bcrypt** password hashing, **Zod** validation on every input, a 1 MB request-body cap, and request logging (morgan).

Security control	Implementation
Authentication	JWT access token (15 min) + refresh token (7 days); refresh tokens tracked in DB; <code>authGuard</code> middleware on admin routes
Password storage	<code>bcryptjs</code> hashing (never stored in plain text)
Input validation	Zod schemas server-side + shared validators client-side (defense in depth)
Transport / headers	<code>helmet</code> ; HTTPS/TLS terminated at nginx (Let's Encrypt)
CORS	Origin allow-list via <code>CORS_ORIGINS</code>
Abuse protection	<code>express-rate-limit</code> (default 300 req / 15 min), HPP, 1 MB body limit
Logging	<code>morgan</code> (combined format in production)

Performance

Are images optimized through Next.js?

No — Next.js image optimization is off (static export requires it). Images are optimized manually (resize + WebP + lazy-loading + fetch-priority). This was the single biggest performance lever on this project.

Is caching implemented?

- **Static assets & HTML:** the whole public site is static files, so nginx serves them with browser/CDN caching; pages are effectively fully cacheable.
- **Fonts:** self-hosted at build time via `next/font` (no external font round-trip).
- **API responses:** no application-level cache today — public endpoints read PostgreSQL directly. Data volumes are small; if traffic grows, add HTTP `Cache-Control` headers or a short-TTL cache in front of the read endpoints.

What are the current Core Web Vitals?

A baseline Lighthouse audit showed **Performance 80, Accessibility 96, Best-Practices 96, SEO 100**. Performance was image-bound. An optimization pass reduced the home page from **~6.7 MB** → **~2.4 MB** transfer (hero image 1.4 MB → 154 KB; site-wide image weight 34.6 MB → 13.3 MB), which directly improves LCP.

Action for the new team: re-run Lighthouse / PageSpeed Insights against the **live domain** after deploying the optimization branch to capture up-to-date LCP / CLS / INP figures. Field data (CrUX) should be monitored once the site has real traffic.

SEO

Rendering for SEO	SSG — every route is pre-rendered to static HTML, fully crawlable (no JS execution needed to read core content/metadata).
Dynamic titles & descriptions	Yes. Per-page metadata via a shared <code>buildMetadata()</code> helper + root <code>metadata</code> (title, description, canonical, keywords).
Open Graph / Twitter	Yes — OG + Twitter summary-large-image cards with default share image.
Schema markup (structured data)	Yes — JSON-LD <code>LocalBusiness</code> block (name, address, geo, phone, email) injected in the root layout.
XML sitemap	Auto-generated at build time (<code>sitemap.ts</code> → static <code>/sitemap.xml</code>) listing all public routes.
robots.txt	Auto-generated (<code>robots.ts</code>); disallows confirmation/success pages; points to the sitemap.
Canonical URLs	Yes, per page, resolved against the canonical site URL.

One config to set: the canonical production domain lives in `prime-source-code/src/utls/site.ts` (`SITE_URL`). Update it to the final domain before the production build so canonical URLs, OG URLs, sitemap and robots all use the correct host.

2 · Architecture Overview

Three independent applications in one monorepo, deployed to a single VPS behind nginx and supervised by pm2, backed by a managed PostgreSQL database.

Tier	What it is	Tech	Runtime
Public site prime-source-code/	Marketing + booking website	Next.js 15 (App Router, static export), React 19, TypeScript, TailwindCSS, framer-motion	Static files served by nginx
Admin dashboard admin/	Staff management console (bookings, enquiries, CMS, catalog)	Next.js 15 (App Router, <code>basePath /admin</code>), React 19, TypeScript	Node process — pm2 <code>pp-admin</code>
Backend API backend/	REST API + database access + file uploads	Node.js, Express, TypeScript, Prisma ORM, Zod, JWT, Multer, Swagger	Node process — pm2 <code>pp-api</code> (port 5000)
Database	Persistent data store	PostgreSQL (managed)	Managed provider
Deploy tooling deploy/	Deploy script, pm2 config, nginx config, env examples	bash, pm2, nginx, certbot	—

Request flow

- **Visitor** → **website**: nginx serves the pre-built static pages instantly. Client-side JS then calls the API for live data (packages, banners, gallery, availability).
- **Visitor submits a form**: browser → `POST` public API endpoint → Zod validation → PostgreSQL. A reference ID / confirmation is returned.
- **Staff** → **admin**: nginx routes `/admin` to the admin Node app; the admin authenticates (JWT) and calls the guarded admin API to manage records and content.
- **Uploads**: images uploaded via the admin are stored on disk (persistent `uploads/` dir) and served back as absolute URLs.

Routing (path-based, current test setup on prime.adpedia.in)

<code>/</code>	Public website (static)
<code>/admin</code>	Admin dashboard (Node app, pm2 <code>pp-admin</code>)
<code>/backend</code>	REST API (Node app, pm2 <code>pp-api</code>) — health at <code>/backend/health</code> , docs at <code>/backend/api/docs</code>

Production can instead use subdomains (`primepromenade.com` , `admin.` , `api.`) — same code, only nginx config + build-time URLs change. Both layouts are documented in `deploy/DEPLOYMENT.md` .

3 · Folder / Project Structure

prime-clone/	(monorepo root)
├─ prime-source-code/	PUBLIC WEBSITE (Next.js, static export)
│ └─ src/app/	routes (home, gym, spa, pool-booking, conference, cafe, pharmacy, study-centre, vendor, gallery, about, contact, terms ...)
│ │ └─ layout.tsx	root layout, metadata, JSON-LD schema
│ │ └─ sitemap.ts / robots.ts	auto-generated SEO files
│ │ └─ globals.css	Tailwind + design tokens
│ └─ src/components/	Header, Footer, ContactSection, Popup, UI/*
│ └─ src/utils/	api.ts (ENDPOINTS), site.ts (SEO), validation.ts
└─ public/	images, logos, icons (ASSETS/, LOGO/, ICONS/)
└─ next.config.ts	output:"export", images unoptimized
├─ admin/	ADMIN DASHBOARD (Next.js, basePath /admin)
│ └─ src/app/(dashboard)/	bookings, pool-calendar, enquiries, enrollments, content, gallery, popup ...
│ └─ src/components/	Shell, ResourceTable, RecordPanel, managers
│ └─ src/lib/	api.ts (auth client), validation.ts
├─ backend/	REST API (Express + Prisma)
│ └─ src/app.ts	middleware stack (helmet, cors, rate-limit...)
│ └─ src/server.ts	bootstrap / graceful shutdown
│ └─ src/routes/	public.routes.ts, admin.routes.ts
│ └─ src/modules/	one folder per domain (bookings, otp, uploads, dashboard, content, listings ...)
│ │ └─ _shared/	generic resource controller/router + registry
│ └─ src/config/	env, prisma, swagger
│ └─ src/middleware/	authGuard, rateLimit, errorHandler
│ └─ prisma/	schema.prisma, migrations/, seed.ts
└─ deploy/	deploy.sh, ecosystem.config.js (pm2), nginx configs, backend.env.example, DEPLOYMENT.md

KEY FILES A NEW DEVELOPER SHOULD READ FIRST

- `CLAUDE.md` (repo root) — the living architecture / working-notes document.
- `backend/src/app.ts` & `backend/src/modules/_shared/` — how the API is wired.
- `backend/prisma/schema.prisma` — the full data model.
- `prime-source-code/src/utils/api.ts` — the front-end ↔ API contract.
- `deploy/DEPLOYMENT.md` & `deploy/deploy.sh` — how it ships.

4 · Deployment Process

Single VPS · nginx (static serving + reverse proxy + TLS) · pm2 (process manager) · managed PostgreSQL. One command redeploys all three tiers.

One-command redeploy

From the repo checkout on the server (`/opt/prime`):

```
cd /opt/prime
./deploy/deploy.sh          # pull → build all tiers → migrate → restart pm2
```

`deploy.sh` performs, in order:

1. **Git:** `fetch + pull --ff-only` the checked-out branch (refuses a diverged history — prevents conflicts).
2. **Backend:** `npm ci → wipe dist → npm run build → prisma generate → prisma migrate deploy` (idempotent) → optional `--seed` (first deploy only).
3. **Admin:** `npm ci → wipe .next → build with the API base URL + base path`.
4. **Site:** `npm ci → wipe .next out → build → rsync -a --delete out/ → web root`.
5. **pm2:** restart `pp-api` and `pp-admin`.

Conflict-free by design: git is fast-forward-only, DB migrations are additive/idempotent, and stale build chunks are wiped each run (`dist / .next / out`). **Never pass `--seed` on a redeploy** — it re-seeds baseline catalog data.

First-time provisioning (once)

1. VPS (Ubuntu 22.04+): install Node 20+, nginx, pm2, certbot.
2. Create a managed PostgreSQL DB; copy its connection string into `backend/.env`.
3. Point DNS A-records to the VPS.
4. Fill `backend/.env` (DB URL, JWT secrets, CORS, seed admin).
5. Run the backend build + `prisma migrate deploy + npm run seed` (first deploy only).
6. Create the persistent uploads dir and symlink it (survives redeploys).
7. Start pm2 apps, enable pm2 startup, install nginx config, issue TLS certs with certbot.

Full step-by-step for both the path-based test setup and the subdomain production setup is in `deploy/DEPLOYMENT.md`. Deploy environment overrides: `API_BASE_URL`, `ADMIN_BASE_PATH`, `SITE_WEB_ROOT`.

Health checks after deploy

```
curl -s https://<domain>/backend/health # {"success":true,...}
pm2 status                             # pp-api + pp-admin "online"
```

5 · API Documentation

The API self-documents with **Swagger / OpenAPI**:

Interactive UI	<code>/api/docs</code> (on the test setup: <code>https://prime.adpedia.in/backend/api/docs</code>)
Raw OpenAPI JSON	<code>/api/docs.json</code>
Health endpoint	<code>/health</code> → <code>{ success, status, uptime }</code>

Response conventions

- **Public endpoints** return the raw JSON shape the website expects (no envelope).
- **Admin endpoints** return `{ success, message, data, meta? }`; the admin client unwraps `.data / .meta`.

Representative endpoints

Area	Public (website)	Admin (JWT-guarded)
Auth	—	<code>/auth/login</code> , <code>/auth/refresh</code>
Pool bookings	<code>POST /pool-bookings</code> , <code>GET /pool-bookings?date=</code>	<code>/pool-bookings/calendar</code> , <code>/month</code> , CRUD, status
Hall / Spa / Gym	<code>POST /hall-bookings</code> , <code>/spa-bookings</code> , <code>/gym-memberships</code>	<code>list / get / update / status / delete</code>
Enquiries & leads	<code>POST /contact-enquiries</code> , <code>/course-registrations</code> , <code>/vendor-invites</code>	<code>list / manage / export</code>
Catalog	<code>GET /listings?type=</code>	CRUD on listings/packages
CMS / content	<code>GET /banners</code> , <code>/gallery</code> , <code>/site-settings</code>	<code>pages / banners / gallery / settings</code> + <code>/uploads</code>
Dashboard	—	<code>/dashboard</code> (aggregate stats)

Swagger is the **source of truth** for exact request/response schemas — always read it live at `/api/docs` rather than relying on this summary.

6 · Database Schema (High Level)

PostgreSQL via Prisma ORM. Schema in `backend/prisma/schema.prisma` ; migrations are hand-written, additive and idempotent (applied with `prisma migrate deploy`).

AUTH & SYSTEM

Model	Purpose
AdminUser	Staff logins (role-based); passwords bcrypt-hashed
RefreshToken	Issued JWT refresh tokens (rotation/revocation)
OtpVerification	OTP codes (phone verification; currently console mode)
AuditLog	Admin action trail

BOOKINGS, MEMBERSHIPS & LEADS

Model	Purpose & reference ID
PoolBooking	Pool reservations — ref <code>P00L-00001</code> ; two-pool availability + GST/add-on pricing logic
HallBooking	Conference-hall enquiries — ref <code>HALL-00001</code>
SpaBooking	Spa bookings — ref <code>SPA-00001</code>
GymMembership	Gym membership sign-ups
CourseRegistration	Study-centre enrolments — ref <code>ENR-00001</code>
VendorInvite	Vendor / counter applications
ContactEnquiry	Contact-form messages / general leads

CONTENT MANAGEMENT & CATALOG

Model	Purpose
Listing	Catalog items / packages (typed by <code>ListingType</code>)
Page	CMS pages
Banner	Banners / carousel slides (incl. the announcement pop-up via <code>location="popup"</code>)
GalleryImage	Gallery images
SiteSetting	Key/value site config (contact numbers, social links, pop-up behaviour...)

ENUMS

`AdminRole` · `BookingStatus` (PENDING / CONFIRMED / CANCELLED / COMPLETED) · `LeadStatus` (NEW / IN_PROGRESS / RESOLVED / ARCHIVED) · `ListingType` .

Reference IDs are generated by the database (Postgres sequences + `lpad`), not application code — keep that pattern when adding new sequenced references, and keep migrations additive.

7 · Performance Notes

- **Static delivery:** the whole public site is pre-built HTML/CSS/JS — no per-request server work, ideal for caching/CDN.
- **Images (the main lever):** optimized manually (resize + WebP, ~1600 px cap). Below-fold images are lazy-loaded; the hero uses high fetch-priority; the auto-scrolling brand strips use lightweight mobile-sized images.
- **Fonts:** self-hosted via `next/font` (Instrument Sans) — no external font request.
- **JS:** healthy first-load JS (~100–160 KB gzipped/route); icon/animation barrel imports are tree-shaken via `optimizePackageImports`.
- **Recorded gains:** home transfer ~6.7 MB → ~2.4 MB; hero 1.4 MB → 154 KB; total image weight 34.6 MB → 13.3 MB.

These optimizations currently live on a dedicated branch (`claude/perf-optimization`) kept separate from the main working branch so they can be reviewed before merging. Confirm which branch is deployed with the outgoing team.

8 · Third-Party Integrations

Integration	Use	Status / where configured
PostgreSQL (managed)	Primary database	Provider of choice (Neon / Supabase / RDS / DO); <code>DATABASE_URL</code> in <code>backend/.env</code>
Google Fonts (Instrument Sans)	Typography	Self-hosted at build via <code>next/font</code> — no runtime dependency
Google Maps	Location map on Contact	Embedded iframe (no API key required)
SMS OTP — MSG91 / Twilio	Phone verification	Not active — OTP runs in "console" mode; provider keys in <code>.env</code> when enabled
Facebook / Instagram	Social links	URLs in site settings / footer
Payment gateway	—	None. Bookings are enquiries; no online payment is collected
Let's Encrypt (certbot)	TLS certificates	Issued/renewed on the VPS via certbot

9 · Backup & Recovery

Asset	Backup	Recovery
Database	Handled by the managed PostgreSQL provider (automated snapshots / point-in-time, per plan)	Restore from the provider's snapshot; update DATABASE_URL if the endpoint changes
Uploaded files	Stored in a persistent server dir (/var/prime-uploads , symlinked into the backend). Back this directory up separately (cron rsync / object storage)	Restore the directory; the symlink re-exposes the files
Source code	GitHub repository (full history)	Re-clone to /opt/prime and run deploy.sh
Config / secrets	backend/.env is NOT in git — keep a secure copy (password manager / secrets vault)	Recreate .env from the secure copy or from .env.example

Recommended: confirm the DB provider's backup retention, schedule a periodic dump of the uploads directory, and store a current copy of backend/.env in a shared secrets manager.

10 · Access, Credentials & Ownership

Assets the new team needs handed over. For security, actual secrets are **not** printed in this document — they should be transferred via a secrets manager.

Item	What to hand over
Source code	GitHub repository access (AdpediaGitOfficial/prime-claude) — owner/admin transfer
Server	VPS SSH access (host, user, key) and the deploy path /opt/prime
Database	Managed PostgreSQL dashboard login + connection string
Domain / DNS	Registrar + DNS provider access; A-records to the VPS
Backend secrets	A copy of backend/.env : DATABASE_URL , JWT_ACCESS_SECRET , JWT_REFRESH_SECRET , CORS_ORIGINS , PUBLIC_BASE_URL , seed-admin, OTP provider keys
Admin dashboard	Seeded admin login (created on first deploy). Change the password on first sign-in. Additional staff users are managed in-app
Email / social	Ownership of info@primepromenade.com and the Facebook/Instagram accounts

Security first steps for the new owner: rotate the JWT secrets and the seeded admin password, review CORS_ORIGINS , and confirm TLS auto-renewal (certbot) is active.

11 · Local Development Quick-Start

```
# Backend
cd backend && npm install
cp .env.example .env # fill DATABASE_URL, JWT secrets
npx prisma migrate deploy && npm run seed
npm run dev # API on :5000, docs at /api/docs

# Public website
cd prime-source-code && npm install
npm run dev # :3000 (NEXT_PUBLIC_API_BASE_URL → your API)

# Admin dashboard
cd admin && npm install
npm run dev # :4000 (basePath /admin)
```

Verify before committing: npx tsc --noEmit and npm run build must pass for each app; backend has npm run typecheck . The repo's CLAUDE.md holds detailed conventions and gotchas.